1 Modular Arithmetic

In several settings, such as error-correcting codes and cryptography, we sometimes wish to work over a smaller range of numbers. Modular arithmetic is useful in these settings, since it limits numbers to a predefined range $\{0, 1, \ldots, N-1\}$, and wraps around whenever you try to leave this range — like the hand of a clock (where $N = 12$) or the days of the week (where $N = 7$).

Example: Calculating the time When you calculate the time, you automatically use modular arithmetic. For example, if you are asked what time it will be 13 hours from 1 pm, you say 2 am rather than 14. Let's assume our clock displays 12 as 0. This is limiting numbers to a predefined range, $\{0, 1, 2, \ldots, 11\}$. Whenever you add two numbers in this setting, you divide by 12 and provide the remainder as the answer.

If we wanted to know what the time would be 24 hours from 2 pm, the answer is easy. It would be 2 pm. This is true not just for 24 hours, but for any multiple of 12 hours (ignoring the detail of am/pm). What about 25 hours from 2 pm? Since the time 24 hours from 2 pm is still 2 pm, after 25 hours it would be 3 pm. Another way to say this is that we add 1 hour, which is the remainder when we divide 25 by 12.

This example shows that under certain circumstances it makes sense to do arithmetic within the confines of a particular number (12 in this example). That is, we only keep track of the remainder when we divide by 12, and when we need to add two numbers, instead we just add the remainders. This method is quite efficient in the sense of keeping intermediate values as small as possible, and we shall see in later lectures how useful it can be.

More generally, we can define $x \pmod m$ (in words: "x modulo m") to be the remainder r when we divide x by m. I.e., if $x \pmod m \equiv r$, then $x = mq + r$ where $0 \leq r \leq m-1$ and q is an integer. Thus $29 \pmod{12} \equiv 5$ and $13 \pmod 5 \equiv 3$.

Computation

If we wish to calculate $x + y \pmod m$, we would first add $x + y$ and then calculate the remainder when we divide the result by m. For example, if $x = 14$ and $y = 25$ and $m = 12$, we would compute the remainder when we divide $x + y = 14 + 25 = 39$ by 12, and get the answer 3. Notice that we would get the same answer if we first computed $2 \equiv x \pmod{12}$ and $1 \equiv y \pmod{12}$ and added the results modulo 12 to get 3. The same holds for subtraction: $x - y \pmod{12}$ is $-11 \pmod{12}$, which is 1. Again, we could have obtained this directly by simplifying first, i.e., $(x \pmod{12}) - (y \pmod{12}) \equiv 2 - 1 \equiv 1$.

This idea saves us even more effort with multiplication: to compute $xy \pmod{12}$, we could first compute $xy = 14 \times 25 = 350$ and then compute the remainder when we divide by 12, which is 2. Notice that we get the same answer if we first compute $2 \equiv x \pmod{12}$ and $1 \equiv y \pmod{12}$ and simply multiply the results modulo 12.

More generally, while carrying out any sequence of additions, subtractions or multiplications mod m, we get the same answer if we reduce any intermediate results mod m. This can considerably simplify the calculations.

Set Representation

There is an alternative view of modular arithmetic which helps understand all this better. For any integer m, we say that x and y are *congruent modulo m* if they differ by a multiple of m or, in symbols,

$$x \equiv y \pmod{m} \iff m \text{ divides } (x-y).$$

For example, 29 and 5 are congruent modulo 12 because 12 divides $29-5$. We can also write $22 \equiv -2 \pmod{12}$. Notice that x and y are congruent modulo m iff they have the same remainder modulo m; in other words,

$$x \equiv y \pmod{m} \iff x \pmod{m} = y \pmod{m}.$$

What is the set of numbers that are congruent to $0 \pmod{12}$? These are all the multiples of 12: $\{\ldots, -36, -24, -12, 0, 12, 24, 36, \ldots\}$. What about the set of numbers that are congruent to $1 \pmod{12}$? These are all the numbers that give a remainder 1 when divided by 12: $\{\ldots, -35, -23, -11, 1, 13, 25, 37, \ldots\}$. Similarly the set of numbers congruent to $2 \pmod{12}$ is $\{\ldots, -34, -22, -10, 2, 14, 26, 38, \ldots\}$. Notice in this way we get 12 such sets of integers, and every integer belongs to one and only one of these sets.

In general, if we work modulo m, then we get m such disjoint sets whose union is the set of all integers: these are often called *residue classes mod m*. We can think of each set as represented by the unique element it contains in the range $(0, \ldots, m-1)$. The set represented by element i would be all numbers z such that $z = mx + i$ for some integer x. Observe that all of these numbers have remainder i when divided by m; they are therefore congruent modulo m.

We can understand the operations of addition, subtraction and multiplication in terms of these sets. When we add two numbers, say $x \equiv 2 \pmod{12}$ and $y \equiv 1 \pmod{12}$, it does not matter which x and y we pick from the two sets, since the result is always an element of the set that contains 3. The same is true about subtraction and multiplication. It should now be clear that the elements of each set are interchangeable when computing modulo m, and this is why we can reduce any intermediate results modulo m.

Here is a more formal way of stating this observation:

Theorem 6.1. *If $a \equiv c \pmod{m}$ and $b \equiv d \pmod{m}$, then $a+b \equiv c+d \pmod{m}$ and $a \cdot b \equiv c \cdot d \pmod{m}$.*

Proof. We know that $c = a + k \cdot m$ and $d = b + \ell \cdot m$ for integers k, ℓ, so $c + d = a + k \cdot m + b + \ell \cdot m = a + b + (k + \ell) \cdot m$, which means that $a + b \equiv c + d \pmod{m}$. The proof for multiplication is similar and left as an exercise. □

Exercise. Complete the proof of Theorem 6.1 for multiplication.

What this theorem tells us is that we can always reduce any arithmetic expression modulo m to a number in the range $\{0, 1, \ldots, m-1\}$. As an example, consider the expression $(13+11) \cdot 18 \pmod{7}$. Using the above theorem several times we can write:

$$\begin{aligned}(13+11) \cdot 18 &\equiv (6+4) \cdot 4 \pmod{7} \\ &= 10 \cdot 4 \pmod{7} \\ &\equiv 3 \cdot 4 \pmod{7} \\ &= 12 \pmod{7} \\ &\equiv 5 \pmod{7}.\end{aligned}$$

In summary, we can always do basic arithmetic (multiplication, addition, subtraction) calculations modulo m by reducing intermediate results modulo m. (Note that we haven't mentioned division: much more on that later!)

2 Exponentiation

Another standard operation in arithmetic algorithms (used heavily, e.g., in primality testing and RSA) is raising one number to a power modulo another number. I.e., how do we compute $x^y \pmod m$, where x, y, m are natural numbers and $m > 0$? A naïve approach would be to compute the sequence $x \pmod m, x^2 \pmod m, x^3 \pmod m, \ldots$ up to y terms, but this requires time exponential in the number of bits in y. We can do much better using the trick of *repeated squaring*:

```
algorithm mod-exp(x, y, m)
  if y = 0 then return (1)
  else
    z = mod-exp(x, y div 2, m)
    if y (mod 2) ≡ 0 then return (z * z (mod m))
    else return (x * z * z (mod m))
```

This algorithm uses the fact that any $y > 0$ can be written as $y = 2a$ or $y = 2a+1$, where $a = \lfloor \frac{y}{2} \rfloor$ (which we have written as `y div 2` in the above pseudo-code), plus the facts

$$x^{2a} = (x^a)^2; \qquad \text{and}$$
$$x^{2a+1} = x \cdot (x^a)^2.$$

Exercise. Use the above facts to prove by induction on y that the algorithm always returns the correct value.

What is its running time? The main task here, as is usual for recursive algorithms, is to figure out how many recursive calls are made. But we can see that the second argument, y, is being (integer) divided by 2 in each call, so the number of recursive calls is exactly equal to the number of bits, n, in y. (The same is true, up to a small constant factor, if we let n be the number of decimal digits in y.) Thus, if we charge only constant time for each arithmetic operation (`div`, `mod` etc.) then the running time of `mod-exp` is $O(n)$. Note that this is *very* efficient: it means that we can handle exponents with (at least) thousands of bits!

In a more realistic model (where we count the cost of operations at the bit level), we would need to look more carefully at the cost of each recursive call. Note first that the test on y in the `if`-statement just involves looking at the least significant bit of y, and the computation of $\lfloor \frac{y}{2} \rfloor$ is just a shift in the bit representation. Hence each of these operations takes only constant time. The cost of each recursive call is therefore dominated by the mod operation[1] in the final result. A fuller analysis of such algorithms is performed in CS170.

3 Bijections

Before talking about division, we are going to have to take a little detour to talk about inverses. From linear algebra and calculus, you already have a lot of intuition about when inverses do and do not exist. Recall that a square matrix has an inverse only if it does not have a non-trivial nullspace. Why? Because if it has a non-trivial nullspace, it maps lots of vectors to the zero vector and there is no way to recover the information that was lost. The concept of bijection just allows us to formalize the mathematical intuition that we already have.

[1] You may want to analyze grade-school long-division for binary numbers to understand how long a mod operation would take. Since all arithmetic is being done mod m, the cost of this operation depends only on the number of bits in m and x (and not on y).

A function is a mapping from a set (called the *domain*) of inputs A to a set of outputs B: for input $x \in A$, $f(x)$ must be in the set B. To denote such a function, we write $f : A \to B$.

Consider the following examples of functions, where both functions map $\{0, \ldots, m-1\}$ to itself:

$$f(x) \equiv x+1 \pmod{m}$$
$$g(x) \equiv 2x \pmod{m}$$

A *bijection* is a function for which every $b \in B$ has a unique *pre-image* $a \in A$ such that $f(a) = b$. Note that this consists of two conditions:

1. f is *onto*: every $b \in B$ has a pre-image $a \in A$.
2. f is *one-to-one*: for all $a, a' \in A$, if $f(a) = f(a')$ then $a = a'$.

Looking back at our examples, we can see that f is a bijection; the unique pre-image of y is $y-1$. However, g is only a bijection if m is odd. Otherwise, it is neither one-to-one nor onto. The following lemma can be used to prove that a function is a bijection:

Lemma: For a finite set A, $f : A \to A$ is a bijection if there is an *inverse* function $g : A \to A$ such that $\forall x \in A$ $g(f(x)) = x$.

Proof. If $f(x) = f(x')$, then $x = g(f(x)) = g(f(x')) = x'$. Therefore, f is one-to-one. Since f is one-to-one, there must be $|A|$ elements in the range of f. This implies that f is also onto. (Can you see why? Prove this last fact for yourself. What kind of proof should you try?) □

The finiteness of the sets involved here make our life easier.

4 Inverses

We have so far discussed addition, multiplication and exponentiation. Subtraction is the inverse of addition and just requires us to notice that subtracting b modulo m is the same as adding $-b \equiv m - b \pmod{m}$.

What about division? This is a bit harder. Over the reals dividing by a number x is the same as multiplying by $y = 1/x$. Here y is that number such that $x \cdot y = 1$. Of course we have to be careful when $x = 0$, since such a y does not exist. Similarly, when we wish to divide by $x \pmod{m}$, we need to find $y \pmod{m}$ such that $x \cdot y \equiv 1 \pmod{m}$; then dividing by x modulo m will be the same as multiplying by y modulo m. Such a y is called the *multiplicative inverse* of x modulo m. In our present setting of modular arithmetic, can we be sure that x has an inverse mod m, and if so, is it unique (modulo m) and can we compute it?

As a first example, take $x = 8$ and $m = 15$. Then $2x = 16 \equiv 1 \pmod{15}$, so 2 is a multiplicative inverse of 8 mod 15. As a second example, take $x = 12$ and $m = 15$. Then the sequence $\{ax \pmod{m} : a = 1, 2, 3, \ldots\}$ is periodic, and takes on the values $(12, 9, 6, 3, 0, 12, 9, 6 \ldots)$. [Exercise: check this!] Thus 12 *has no multiplicative inverse mod 15* since the number 1 never appears in this sequence.

This is the first warning sign that working in modular arithmetic might actually be very different from grade-school arithmetic. Two weird things are happening. First, no multiplicative inverse seems to exist for a number that isn't zero. (In normal arithmetic, the only number that has no inverse is zero.) Second, the "times table" for a number that isn't zero has zero showing up in it! So, e.g., 12 times 5 is equal to zero when we are considering numbers modulo 15. (In normal arithmetic, zero never shows up in the multiplication table for any number other than zero.)

So, when *does* x have a multiplicative inverse modulo m? The answer is: if and only if the greatest common divisor of m and x is 1. Moreover, when the inverse exists it is unique. Recall that the *greatest common divisor* of two natural numbers x and y, denoted $\gcd(x,y)$, is the largest natural number that divides them both. For example, $\gcd(30,24) = 6$. If $\gcd(x,y)$ is 1, it means that x and y share no common factors (except 1). This is often expressed by saying that x and y are *relatively prime* or *coprime*.

Theorem 6.2. *Let m,x be positive integers such that $\gcd(m,x) = 1$. Then x has a multiplicative inverse modulo m, and it is unique (modulo m).*

Proof. Consider the sequence of m numbers $0, x, 2x, \ldots (m-1)x$. We claim that these are all distinct modulo m. Since there are only m distinct values modulo m, it must then be the case that $ax \equiv 1 \pmod{m}$ for exactly one a (modulo m). This a is the unique multiplicative inverse of x.

To verify the above claim, suppose for contradiction that $ax \equiv bx \pmod{m}$ for two distinct values a,b in the range $0 \leq b \leq a \leq m-1$. Then we would have $(a-b)x \equiv 0 \pmod{m}$, or equivalently, $(a-b)x = km$ for some integer k (possibly zero or negative).

However, x and m are relatively prime, so x cannot share any factors with m. This implies that $a-b$ must be an integer multiple of m. This is not possible, since $a-b$ ranges between 1 and $m-1$. □

Actually it turns out that $\gcd(m,x) = 1$ is also a *necessary* condition for the existence of an inverse: i.e., if $\gcd(m,x) > 1$ then x has no multiplicative inverse modulo m.

Exercise. Verify this claim. [Hint: Assume for contradiction that x does have an inverse, say a. Then write down a (non-modular) equation that x, m and a must satisfy. Finally, derive a contradiction from the fact that x and m have a common factor $d > 1$.]

Since we know that multiplicative inverses are unique when $\gcd(m,x) = 1$, we shall write the inverse of x as $x^{-1} \pmod{m}$. Being able to compute the multiplicative inverse of a number is crucial to many applications, so ideally the algorithm used should be efficient. It turns out that we can use an extended version of Euclid's algorithm, which computes the gcd of two numbers, to compute the multiplicative inverse.

5 Computing Inverses: Euclid's Algorithm

Let us first discuss how computing the multiplicative inverse of x modulo m is related to finding $\gcd(x,m)$. For any pair of numbers x,y, suppose we could not only compute $\gcd(x,y)$, but also find integers a,b such that

$$d = \gcd(x,y) = ax + by. \tag{1}$$

(Note that this is not a modular equation; and the integers a,b could be zero or negative.) For example, we can write $1 = \gcd(35,12) = -1 \cdot 35 + 3 \cdot 12$, so here $a = -1$ and $b = 3$ are possible values for a,b.

If we could do this then we'd be able to compute inverses, as follows. We first find integers a and b such that

$$1 = \gcd(m,x) = am + bx.$$

But this means that $bx \equiv 1 \pmod{m}$, so b is a multiplicative inverse of x modulo m. Reducing b modulo m gives us the unique inverse we are looking for. In the above example, we see that 3 is the multiplicative inverse of 12 mod 35. So, we have reduced the problem of computing inverses to that of finding integers a,b that satisfy (1). Remarkably, Euclid's algorithm for computing gcd's also allows us to find integers a and b

as described above. So computing the multiplicative inverse of x modulo m is as simple as running Euclid's gcd algorithm on input x and m!

Euclid's Algorithm

If we wish to compute the gcd of two numbers x and y, how would we proceed? If x or y is 0, then computing the gcd is easy; it is simply the other number, since 0 is divisible by everything (although of course it divides nothing). The algorithm for other cases is ancient, and although associated with the name of Euclid, is almost certainly a folk algorithm invented by craftsmen (the engineers of their day) because of its intensely practical nature[2].

This algorithm exists in cultures throughout the globe.

The algorithm for computing $\gcd(x,y)$ uses the following theorem to eventually reduce to the case where one of the numbers is 0.

Theorem 6.3. *Let $x \geq y > 0$. Then $\gcd(x,y) = \gcd(y, x \pmod y)$.*

Proof. The theorem follows immediately from the fact that a number d is a common divisor of x and y *if and only if* d is a common divisor of y and $x \pmod y$. To see this, write $x = qy + r$ where q is an integer and $r = x \pmod y$. Then, if d divides x and y then it also divides x and qy, and thus it also divides their difference $r = x - qy$ (as we proved in Note 1). Conversely, if d divides y and r then it also divides qy and r and thus also their sum $x = qy + r$. □

Given this theorem, let's see how to compute $\gcd(16,10)$:

$$\begin{aligned}\gcd(16,10) &= \gcd(10,6)\\ &= \gcd(6,4)\\ &= \gcd(4,2)\\ &= \gcd(2,0) = 2\end{aligned}$$

In each line, we replace the pair of arguments (x,y) with $(y, x \pmod y)$, until the second argument becomes 0. At this point the gcd is just the first argument. By the theorem, each of these substitions preserves the gcd.

This algorithm can be written recursively as follows. The algorithm assumes that the inputs are natural numbers x, y satisfying $x \geq y \geq 0$ and $x > 0$.

```
algorithm gcd(x, y)
  if y = 0 then return(x)
  else return(gcd(y, x (mod y)))
```

Theorem 6.4. *The algorithm above correctly computes the gcd of x and y.*

[2] This algorithm is used for figuring out a common unit of measurement for two lengths. You can imagine how this is extremely important for building something up from a scale model. Different lengths in a design can be expressed as integer multiples of a common length, and then a new measuring stick can be found for the scaled-up design. The fact that Euclid's algorithm deals naturally with real numbers is also important in understanding the topic of continued fractions — a topic important in the understanding of approximations and numerical computing.

Proof. Correctness is proved by (strong) induction on y, the smaller of the two input numbers. For each $y \geq 0$, let $P(y)$ denote the proposition that the algorithm correctly computes $\gcd(x,y)$ for all values of x such that $x \geq y$ (and $x > 0$). Certainly $P(0)$ holds, since $\gcd(x,0) = x$ and the algorithm correctly computes this in the `if`-clause. For the inductive step, we may assume that $P(z)$ holds for all $z < y$ (the inductive hypothesis); our task is to prove $P(y)$. The key observation here is that $\gcd(x,y) = \gcd(y, x \pmod y)$ — that is, replacing x by $x \pmod y$ does not change the gcd. This was proved in Theorem 6.3. Hence the `else`-clause of the algorithm will return the correct value provided the recursive call `gcd(y,x mod y)` correctly computes the value $\gcd(y, x \pmod y)$. But since $x \pmod y < y$, we know this is true by the inductive hypothesis! This completes our verification of $P(y)$, and hence the induction proof. □

What is the running time of this algorithm? We shall see that, in terms of arithmetic operations on integers, it takes time $O(n)$, where n is the total number of bits in the input (x,y). This is again very efficient. The argument for this fact will be similar to the one we used earlier for exponentiation, but slightly trickier: it is obvious that the arguments of the recursive calls become smaller and smaller (because $y \leq x$ and $x \pmod y < y$). The question is, how fast?

The key point we will prove is that, in the computation of $\gcd(x,y)$, after *two* recursive calls the first (larger) argument is smaller than x by at least a factor of two (assuming $x > 0$). (Note that we can't argue much about what happens in just one call.) There are two cases:

1. $y \leq \frac{x}{2}$. Then the first argument in the next recursive call, y, is already smaller than x by a factor of 2, and thus in the next recursive call it will be even smaller.
2. $x \geq y > \frac{x}{2}$. Then in two recursive calls the first argument will be $x \pmod y$, which is smaller than $\frac{x}{2}$.

So, in both cases the first argument decreases by a factor of at least two every two recursive calls. Thus after at most $2n$ recursive calls, where n is the number of bits in x, the recursion must stop. (Note that the first argument is always a natural number.)

Note that the above argument only shows that the *number of recursive calls* in the computation is $O(n)$. We can make the same claim for the running time if we assume that each call only requires constant time. Since each call involves one integer comparison and one mod operation, it is reasonable to claim that its running time is constant. In a more realistic model of computation, however, we should really make the time for these operations depend on the size of the numbers involved. This will be discussed in CS170.

Extended Euclid's Algorithm

Recall that, in order to compute the multiplicative inverse, we need an algorithm which also returns integers a and b such that:

$$\gcd(x,y) = ax + by. \tag{2}$$

Then, in particular, when $\gcd(x,y) = 1$ we can deduce that b is an inverse of y mod x.

Now since this problem is a generalization of the basic gcd, it is perhaps not too surprising that we can solve it with a fairly straightforward extension of Euclid's algorithm.

The following recursive algorithm `extended-gcd` follows the same recursive structure as Euclid's original algorithm, but keeps track of the required coefficients a, b in (2) as the recursion unwinds. Specifically, the algorithm takes as input a pair of natural numbers $x \geq y$ as in Euclid's algorithm, and returns a triple of integers (d,a,b) such that $d = \gcd(x,y)$ and $d = ax + by$:

```
algorithm extended-gcd(x, y)
```

```
if y = 0 then return(x, 1, 0)
else
  (d, a, b) := extended-gcd(y, x (mod y))
  return((d, b, a - (x div y) * b))
```

In this algorithm, x div y denotes the usual truncated integer division, written mathematically as $\lfloor x/y \rfloor$.

Here is the sequence of recursive calls (top row) along with the sequence of triples that they return (bottom row) for the same input $(x,y) = (16,10)$ as in our previous gcd example:

$$\begin{array}{ccccccccc} \text{egcd}(16,10) & \longrightarrow & \text{egcd}(10,6) & \longrightarrow & \text{egcd}(6,4) & \longrightarrow & \text{egcd}(4,2) & \longrightarrow & \text{egcd}(2,0) \\ (2,2,-3) & \longleftarrow & (2,-1,2) & \longleftarrow & (2,1,-1) & \longleftarrow & (2,0,1) & \longleftarrow & (2,1,0) \end{array}$$

Exercise. Check the above execution sequence.

The final triple returned is $(d,a,b) = (2,2,-3)$, meaning that the gcd of $(x,y) = (16,10)$ is $d = 2$, and that it can be expressed as $d = ax + by = 2 \cdot 16 - 3 \cdot 10$, which we can easily see is correct. (In fact, the sequence of triples (d,a,b) returned each expresses $d = 2$ as a linear combination of the corresponding inputs to that recursive call: so we can check that $d = 1 \cdot 2 + 0 \cdot 0 = 0 \cdot 4 + 1 \cdot 2 = 1 \cdot 6 - 1 \cdot 4 = -1 \cdot 10 + 2 \cdot 6 = 2 \cdot 16 - 3 \cdot 10$.) Since $\gcd(16,10) \neq 1$, 10 doesn't have an inverse mod 16, so the coefficients a,b returned by the algorithm are not useful to us for computing inverses. However, the next exercise suggests an example where they allow us to read off the inverse, as described earlier.

Exercise. Hand-turn the algorithm yourself on the input $(x,y) = (35,12)$ and verify that it returns the triple $(1,-1,3)$. Deduce that the inverse of 12 mod 35 is 3.

Let's now reverse-engineer the algorithm and understand why it works and why it was designed this way. In the base case ($y = 0$), the algorithm returns the gcd value $d = x$ as before, together with coefficients $a = 1$ and $b = 0$; clearly these satisfy $ax + by = d$, as required.

When $y > 0$, the algorithm first recursively computes values (d,a,b) such that $d = \gcd(y, x \pmod y)$ and

$$d = ay + b(x \pmod y). \tag{3}$$

It then returns the triple (d,A,B), where $A = b$ and $B = a - \lfloor x/y \rfloor b$. Just as in our earlier analysis of the vanilla gcd algorithm, we know that the value d computed recursively will be equal to $\gcd(x,y)$. So the first component of the triple returned by the algorithm is correct.

What about the other two components, A and B? From the specification of the algorithm, they should be integers that satisfy

$$d = Ax + By. \tag{4}$$

To figure out what A and B should be in terms of the previously returned values a and b, we can rearrange (3), as follows:

$$\begin{aligned} d &= ay + b(x \pmod y) \\ &= ay + b(x - \lfloor x/y \rfloor y) \\ &= bx + (a - \lfloor x/y \rfloor b)y. \end{aligned} \tag{5}$$

(In the second line here, we have used the fact that $x \pmod{y} = x - \lfloor x/y \rfloor y$ — check this!) Now compare (5) with (4): comparing coefficients of x and y, we see that we need to take $A = b$ and $B = a - \lfloor x/y \rfloor b$. But this is exactly what the last line of the algorithm does, and this is why it works!

Exercise. Turn the above argument into a formal proof by induction that the algorithm extended-gcd is correct.

Since the extended gcd algorithm has exactly the same recursive structure as the vanilla version, its running time will be the same up to constant factors (reflecting the increased time per recursive call). So once again the running time on n-bit numbers will be $O(n)$ arithmetic operations. This means that we can find multiplicative inverses very efficiently.

Remark.

We note that a different version of the algorithm which is easier to run by hand can be developed as follows. The goal is to find $ax+by = d = gcd(x,y)$. We first observe if $d \mid x$ (or $x = id$) and $d \mid y$ (or $y = jd$), we have

$$ax + by = a(id) + b(jd) = (ai + bj)d,$$

or the expression $ax+by$ must be a multiple of the $gcd(x,y)$. Thus, we simply want to find the equation of this form with the smallest strictly positive right hand side.

We begin with the following identities of this form as follows.

$$(1)x + (0)y = x$$
$$(0)x + (1)y = y$$

Again the right hand sides are multiples of $d = gcd(x,y)$ as are any sum of integer multiples of the two equations. We wish to make the right hand sides "smaller": one way to do this is to subtract the second equation from the first, or more aggressively subtracting a multiple of the second from the first. In an example with $x = 16$ and $y = 6$, we have

$$(1)16 + (0)6 = 16$$
$$(0)16 + (1)6 = 6$$

and now subtracting two ($\lfloor 16/6 \rfloor$) times the second from the first, we obtain

$$(1)16 + (-2)6 = 4.$$

We can then subtract this equation from the previous, and obtain

$$(-1)16 + (3)6 = 2.$$

Now, we have obtained a solution as the right hand side is $gcd(16,6)$. In general, if one observes that if the previous equations have a right hand sides of x (e.g., 16) and y (e.g., 6) , the successive equation has a right hand side of $x - \lfloor x/y \rfloor y$ (e.g., $16 - (2)(6) = 4$) akin to the recursive call in the gcd algorithm. Moreover, the coefficients of the left hand side are the A, B pair that are returned by the e-gcd algorithm called with x and y. This method is essentially an iterative version of the e-gcd algorithm.

Division in modular arithmetic

Now that we know how to compute the inverse of x modulo m (assuming that x and m are coprime), how can we use it to do arithmetic? The simplest scenario is solving a modular equation such as the following:

$$8x \equiv 9 \pmod{15}. \tag{6}$$

To solve the analogous equation $8x = 9$ over the rational numbers, we would multiply both sides by 8^{-1} to get $x = 9/8$. Let's do the same thing in arithmetic mod 15. Recall that the inverse of $8 \pmod{15}$ is 2 (since $2 \cdot 8 = 16 \equiv 1 \pmod{15}$). Hence we can multiply both sides of (6) by $8^{-1} \equiv 2$ to get

$$x \equiv 18 \equiv 3 \pmod{15}.$$

i.e., the solution to the modular equation in (6) is $x = 3$, and this solution is unique modulo 15.

6 Fundamental Theorem of Arithmetic

You may recall that any positive integer greater than 1 can be expressed uniquely as a product of primes, up to a reordering of the factors. This is known as the Fundamental Theorem of Arithmetic. For example, $12 = 2 \cdot 2 \cdot 3$. How do we know that this is true? It turns out that Extended Euclid's Algorithm is the key to proving this!

We'll start by proving an important fact about divisibility.

Claim: Let x, y, and z be positive integers such that $\gcd(x,y) = 1$. If $x \mid yz$, then $x \mid z$.

Proof: By Extended Euclid's algorithm, since $\gcd(x,y) = 1$, there exists integers a and b such that

$$1 = \gcd(x,y) = ax + by.$$

Multiplying both sides by z gives

$$z = axz + byz.$$

We know that $x \mid axz$, and $x \mid byz$ because $x \mid yz$. Thus, x divides their sum, or in other words, $x \mid axz + byz$. Since $axz + byz = z$, we conclude that $x \mid z$. □

Recall that a prime number p is a number whose only positive factors are 1 and p.

Fundamental Theorem of Arithmetic: Every positive integer $n > 1$ can be expressed uniquely in the form $p_1 p_2 \cdots p_k$, where each p_i is a (not necessarily unique) prime number, up to reordering of the prime factors.

For example, we can write $12 = 2 \cdot 2 \cdot 3$. Any other prime factorization of 12 is some reordering of $2, 2$, and 3.

Proof: We have shown in lecture that every positive integer n can be expressed in the form $p_1 p_2 \cdots p_k$, where each p_i is a (not necessarily unique) prime number. Thus, it suffices to show that such a form is unique up to reordering the factors.

In other words, suppose we can express n with two prime factorizations

$$n = p_1 p_2 \cdots p_k = q_1 q_2 \cdots q_l,$$

where the p_i and q_j are prime numbers. We need to show that $k = l$, and the p_i's are some reordering of the q_j's.

Without loss of generality, assume that $k \leq l$. (If $k \geq l$, we can simply swap the factorizations.) Our goal is to show that each p_i is one of the q_j's.

First, consider p_1. Since $p_1 \mid n$, we must have $p_1 \mid q_1 q_2 \cdots q_l$. If p_1 is one of $q_1, q_2, \ldots, q_{l-1}$, then we are done. Otherwise, if $p_1 \neq q_j$, for $1 \leq j \leq l-1$, then $\gcd(p_1, q_j) = 1$ for $1 \leq j \leq l-1$, as the only factor they share in common is 1. Applying the previous claim, we conclude that $p_1 \mid q_l$. Since 1 is not prime, p_1 is not 1, so $p_1 = q_l$.

Thus, we conclude that $p_1 = q_j$ for some j. We can thus divide both sides by p_1, reducing the number of primes on both sides by 1.

We now repeat the process for p_2, p_3, all the way to p_k. In the end, the left hand side will be 1, and the right hand side will be a product of $l-k$ primes. Every prime number is at least 2, so for the two sides to be equal, we must have $l-k=0$, or $k=l$. Moreover, in this process, we have matched each p_i with a unique q_j. Thus, the p_i's are some reordering of the q_j's, as desired. □

What was the key to the above proof? Well, to prove that a prime factorization existed (something we showed in lecture), we used strong induction. To prove that such a factorization is unique, we used a corollary of the Euclidean algorithm. The key to the Euclidean algorithm is the existence and uniqueness of division with remainder; in other words, for integers x and $m \neq 0$, there exists unique integers q and r such that $x = mq + r$ with $0 \leq r \leq m-1$. If we have other arithmetic systems that emulate this type of division algorithm, we can follow the same type of proof to get a form of unique prime factorization. This is an important generalization in number theory! [3]

7 Chinese Remainder Theorem.

With our understanding of inverses, we can now present an interesting aspect of modular arithmetic which is embodied in the Chinese Remainder Theorem, named thusly since its earliest known statement was by the Chinese mathematician Sunzi in the 3rd century AD.

The simplest case of the theorem is as follows:

Claim: For m, n with $gcd(m,n) = 1$ that there is exactly one $x \pmod{mn}$ that satisfies the equations:

$$x \equiv a \pmod{n} \text{ and } x \equiv b \pmod{m}.$$

The proof follows from the existence of inverses of n and m respectively modulo m and n, which holds when $\gcd(n,m) = 1$.

Proof: Let $u \equiv m(m^{-1} \pmod{n})$ and $v \equiv n(n^{-1} \pmod{m})$.

Note that

$$u \equiv 1 \pmod{n} \quad \text{and} \quad u \equiv 0 \pmod{m}$$

since $m(m^{-1} \pmod{n}) \equiv mm^{-1} \equiv 1 \pmod{n}$ and $mx \equiv 0 \pmod{m}$ for any x, including $x \equiv m^{-1} \pmod{n}$.

Similarly, we have

$$v \equiv 0 \pmod{n} \quad \text{and} \quad v \equiv 1 \pmod{m}$$

Thus, $x = ua + vb \equiv a(1) + b(0) \equiv a \pmod{n}$, and similarly $x \equiv b \pmod{m}$. That is we have an x that satisfies the equations.

[3] If you are interested in seeing more details, take a look at https://en.wikipedia.org/wiki/Euclidean_domain.

To argue that there is a unique solution, consider two solutions x and y in $\{0,\ldots,mn-1\}$. Now $x-y \equiv 0 \pmod{n}$ and $x-y \equiv 0 \pmod{m}$, which implies that $x-y$ is a multiple of m and n. Since $gcd(m,n)=1$, we have that $x-y=kmn$ for some integer k, which implies that $|x-y|=0$ or $|x-y| \geq mn$. The former implies that $x=y$, the latter implies that one of x or y is not in $\{0,\ldots,mn-1\}$. That is, there is at most one solution in $\{0,\ldots,mn-1\}$. □

A quick review is that we formed a solution by finding u where $u \equiv 0 \pmod{n}$, and a v where $v \equiv 0 \pmod{m}$, which can then be added to together to get a solution for each modulus as the 0 doesn't affect the equations in the other modulus. The uniqueness follows since the difference of any two solutions has a difference which is a multiple of mn.

This can be extended to the full Chinese remainder theorem which states.

Chinese Remainder Theorem: Let $n_1, n_2, \ldots, n_k$ be positive integers that are coprime to each other. Then, for any sequence of integers a_i there is a unique integer x between 0 and $N = \prod_{i=1}^{k} n_i$ that satisfies the congruences:

$$\begin{cases} x \equiv a_1 \pmod{n_1} \\ \vdots \equiv \vdots \\ x \equiv a_i \pmod{n_i} \\ \vdots \equiv \vdots \\ x \equiv a_k \pmod{n_k} \end{cases}$$

Moreover,

$$x \equiv \left(\sum_{i=1}^{k} a_i b_i \right) \pmod{N} \tag{7}$$

where $b_i = \frac{N}{n_i} \left(\frac{N}{n_i} \right)_{n_i}^{-1}$ where $\left(\frac{N}{n_i} \right)_{n_i}^{-1}$ denotes the multiplicative inverse $\pmod{n_i}$ of the integer $\frac{N}{n_i}$.

Here, each b_i is $1 \pmod{n_i}$ and $0 \pmod{n_j}$ for $i \neq j$, and thus a multiple of b_i can be used to satisfy the equation $\pmod{n_i}$ while not affecting the solution of the equations $\pmod{n_j}$ for $i \neq j$. It may be useful to think of each x as the k-dimensional vector where the ith entry is $x \pmod{n_i}$. With this view, b_i is the vector consisting of 1 in the ith entry and 0 elsewhere. Then one can obtain the vector for x by adding multiples of the "basis" vectors, b_i.

The uniqueness follows again from the fact that the difference of two solutions, x and y must be a multiple of $N = n_1 \times \cdots \times n_k$ since they are relatively prime and therefore there cannot be two of them in the range $\{0,\ldots,\prod_{i=0}^{k} n_i - 1\}$. Or briefly, notice that for any two solutions, x, y, have $(x-y) \equiv 0 \pmod{n_i}$ for all i which implies the difference is a multiple of all the n_i and therefore larger than $\prod_{i=0}^{k} n_i$ which is the desired contradiction.

Finally, we note that x can be represented by (a,b) where $x \equiv a \pmod{n}$ and $x \equiv b \pmod{m}$, and given x and y with representations (x_a, x_b) and (y_a, y_b), that $x+y$ has the representation (x_a+y_a, x_b+y_b). A similar observation works for multiplication, which implies that arithmetic $\pmod{mn}$ acts in the same manner as arithmetic over the pairs of numbers viewed in modulo m and n respectively. Thus, in addition to a bijection between these sets, this mapping between $\{0,\ldots,mn-1\}$ and $\{0,\ldots,n-1\} \times \{0,\ldots,m-1\}$ are considered isomorphic under these operations as well. This relationship extends in the same manner to a case where there are more than two relatively prime moduli.